DevOps Introduction

Óscar Barrios
Test Automation Specialist @SUSE

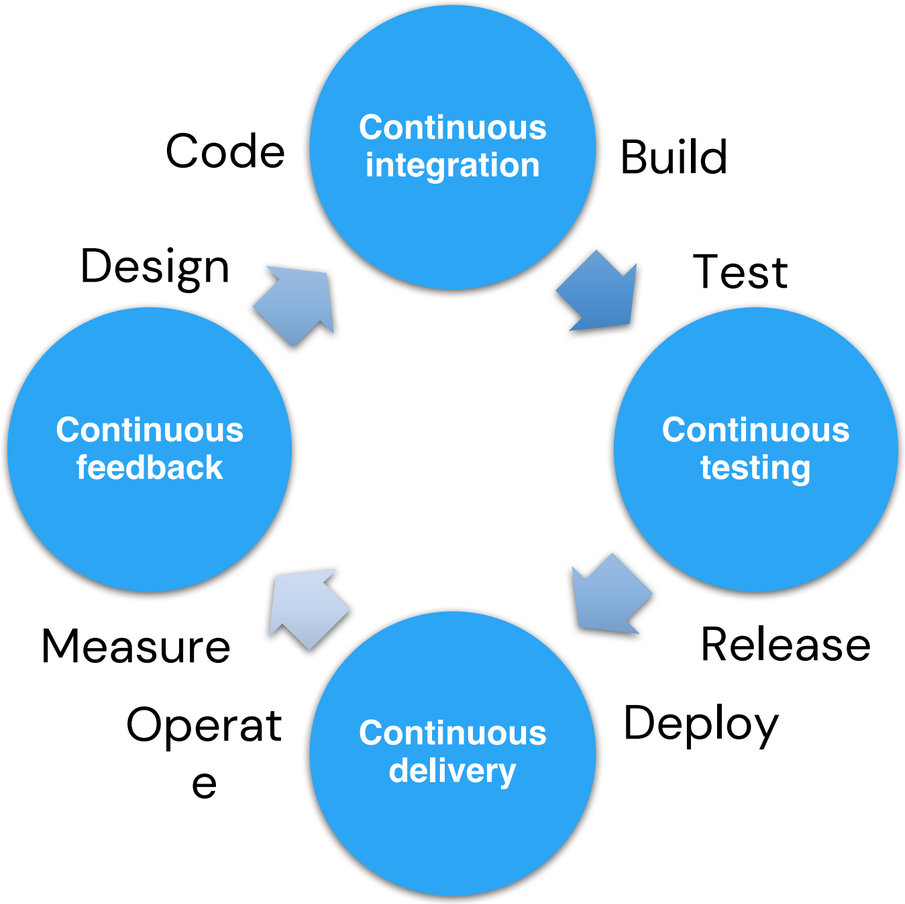
1 DevOps culture

2 DevOps cycle

3 Practical case study

DevOps Cycle

- Design
- Code
- Build
- Test
- Release
- Deploy
- Operate
- Measure



DevOps is a culture that fosters cross-functional team communication

DevOps principles

- **Cross-functional** team communication
- DevOps **expanded** the whole team **responsibility** for delivering **quality** software
- **Automation** of processes
- **Measurement** of KPIs
- Sharing **feedback**, best practices, and knowledge

Continuous Integration

- **Integrate code** into a shared repository one or more times per day
- **Each commit verified** by **FAST** automated tests
  - Static code analysis, Unit Testing, API testing
- If a test fail, we warn the commiter about the issue
- If the tests pass successfully, we can **generate a new build**
  - This step is optional, but mandatory to start the next stage "Continous Testing"
  - Consider Infrastructure as Code (automated deployments)

Continuous Testing

- **Validate** a release **candidate**
  - *Integration tests (often part of CI)*
  - **End-to-end**/System/Acceptance tests
- Increase **test coverage**
  - Improves product quality
- We need to deploy a Test environment
- Not all is automated!
  - In case of errors, review the **Test report**
  - **Exploratory testing** of new feature
- Accelerate testing phace thanks to **parallelism**
- Integrates well with methodologies to approach tests to non-technical people
  - **Behaviour Driven Development**
- **Transparent communication**
  - Drops silos between development and testing

Continuous Delivery

- Ability to **get changes** of new features, configuration, bug fixes and experiments **into production**, *safely* and *quickly* in a *sustainable* way.
- **Each commit** is verified as deployable **release candidate**
- Deploying is a manual step
  - **Continuous Deployment** has the same workflow but automatically deploying to production

Continuous Delivery

- Help to achieve these goals:
  - Faster time to market
  - Better products

    Techniques such as A/B testing enable us to take a hypothesis-driven approach, testing ideas with users before building out whole features.

    This means we can avoid features we build that does not deliver value to our businesses.
  - Happier teams

    Release to production is routine and stress-free

Continuous Feedback

At this stage we want to **detect problematic areas of a process** and **analyze the feedback from the users**, reporting these problems to the beginning of the Development cycle.

Logging, Monitoring, Alerting, Observability

- Infrastructure
- Services (i.e. logs, alarms, downtime)
- KPIs
- User Experience (I.e. tracking user behaviour patterns)
- Bug reports
- Others

Testing in production

- Chaos engineering
- A/B Testing

Uyuni as practical case study

Continuous Integration

- **Jenkins** as automation server for CI
- **GitHub** as Git Repository in a public account
  - We have are migrating to GitHub Actions for our unit tests
- **Gitarro** (custom tool) to poll repository commit changes
  - If offers a special filter to skip concrete unit tests
- Unit & Integration tests as Jenkins Jobs (not pipelines) using Gitarro
  - **None of the tests requires a deployment** of a test environment, they run in a Jenkins Slaves

Continuous Integration

- We name these tests **Pull Request Checks**, as the tests are triggered only in a Pull Request scope, not in direct commits.
    - We can **block PR merge**, depending on the failed PR check
- PR Checks are not immediate, due to the polling mechanism
    - We started a transition to GitHub Actions in Uyuni, see here an example
    - Still a possibility to migrate our private repository to GitHub Actions, by using Self-hosted runners

Continuous Testing

- We have an automated **End to End test suite**, which runs **WebUI** and **command** testing
- It is coded in top of a Test Framework written in **Ruby** following **BDD** using **Cucumber** framework
  - You can have an overview of BDD and project structure here
- On Jenkins we have a Jenkins Pipeline for each supported version running triggered every 4 hours
- We need to deploy a test environment for each testsuite run
  - We use **Sumaform** **(Terraform + Salt)** to deploy the environment in **virtual machines**
  - We need to use virtual machines as SUMA needs to connect to clients on different operating systems

Continuous Testing

- We use a tool named Terracumber to handle the set of environments within the Sumaform tool (deploy) and the execution of Cucumber tool (test)
- The TestSuite is split in stages, by priority, so **if a stage fails it aborts the next stage**
- UI tests use to be **very very slow**! So if possible, avoid to have a huge testsuite of UI tests
  - If you start a project with UI tests, start by adding **parallel test support**, even if you don't have many, otherwise it's harder to refactor later
- Be caution with **idempotency issues**, if you run all the tests reusing the same environment

Continuous Testing

- If the test suite fails
  - We need to manually review the Test Report, see here an example.
  - The test report is generated using a cucumber-html-reporter
  - We also store as artifact a compressed folder with server files useful for debugging
  - We sent an e-mail with a testsuite summary to warn everyone about these failures
  - We *don't* block next testsuite execution, so we might lose the test environment for debug the issue

Continuous Delivery

- It is hard to have in SUSE Manager, due to our current development cycle
- What about our upstream Uyuni?

Continuous Feedback

- For now, we only have in place Bugzilla as bug reporter
- We considered to have some feedback sender integrated on the WebUI
- We monitor a SUSE Manager server that is in "production" internally
  - Handling our Development environments